CC ASSIGNMENT 3

Bottom-Up Parsers

LR(1) & SLR(1)

23i-0832 Humayun Bilal

23i-0728 M.Faheem

1.Introduction

Bottom-up parsing builds a parse by repeatedly reducing substrings of input to grammar non-terminals until the start symbol is produced. LR-family parsers are deterministic bottom-up parsers that use parsing tables and a stack of states/symbols.

This project implements two LR parsers for context-free grammars:

1. SLR(1), which uses LR(0) items plus FOLLOW sets for reduce actions.
2. LR(1), which uses LR(1) items with explicit lookahead per item.

Goal of the project:

1. Build canonical item collections and parsing tables for both methods.
2. Parse multiple input strings with full step traces.
3. Generate parse trees for accepted strings.
4. Compare parser power, number of states, table size, conflicts, and runtime behavior.
5. Approach

2.1 Data Structures Used

Production

Stores production id, left-hand side non-terminal, and right-hand side symbol list.

Supports epsilon productions as empty right-hand side.

LR(0) Item

Stores production plus dot position.

Used by SLR(1) canonical collection and table construction.

LR(1) Item

Stores production, dot position, and one lookahead symbol.

Used by LR(1) canonical collection and precise reduce placement.

Parsing Table

ACTION table: terminal-driven shift/reduce/accept actions.

GOTO table: non-terminal transitions.

Conflict list: records shift/reduce or reduce/reduce conflicts with state and symbol.

Parser Runtime Stacks

State stack for LR automaton states.

Symbol stack for grammar symbols.

Parse-tree node stack for building parse trees during reductions.

Parse Tree Node

Tree node with symbol and children.

Constructed during reductions and printed in ASCII form.

2.2 Algorithm Implementation Details

A. Grammar Loading and Augmentation

1. Grammar lines are read and cleaned (comments and blank lines skipped).
2. Original start symbol is taken from first production.
3. Augmented start symbol is generated (StartPrime style).
4. Production 0 is added as augmented rule:
   AugmentedStart -> OriginalStart

B. FIRST and FOLLOW

1. FIRST sets are computed iteratively to fixed point.
2. FOLLOW sets are computed iteratively using FIRST of suffixes and FOLLOW propagation.
3. FIRST of sequence is implemented and reused in LR(1) closure.

C. SLR(1) Canonical Collection

1. Start from closure of augmented item with dot at beginning.
2. For each state and grammar symbol, compute GOTO.
3. Add new states when GOTO result is not already present.

D. SLR(1) Table Construction

1. If item has terminal after dot, place SHIFT.
2. If item has non-terminal after dot, place GOTO.
3. If dot is at end:

- For augmented start production, place ACCEPT on end marker.
- Otherwise place REDUCE on all symbols in FOLLOW(lhs).

E. LR(1) Canonical Collection

1. Start with item [AugmentedStart -> . OriginalStart, $].
2. LR(1) closure expansion:

- For [A -> alpha . B beta, a]
- Add [B -> . gamma, b] for each b in FIRST(beta a)

3. GOTO and state expansion analogous to SLR, but with LR(1) items.

F. LR(1) Table Construction

1. SHIFT and GOTO are filled similarly.
2. If dot at end for non-augmented production, REDUCE is placed only on that item's lookahead.
3. ACCEPT only on end marker for augmented item.

G. Parsing Procedure (Both Parsers)

1. Append end marker to token stream.
2. Loop:

- Read action from ACTION[topState, lookahead].
- SHIFT: push symbol and next state.
- REDUCE: pop rhs length, create parent node lhs, push lhs and goto state.
- ACCEPT: finish with parse tree.
- Missing action: reject with informative error.

3. Save full parsing trace and reduction sequence.

2.3 Design Decisions and Trade-offs

1. Deterministic table-driven parser design

- Clear separation between automaton building and runtime parsing.
- Easier to debug and compare SLR vs LR(1).

2. LinkedHashMap and LinkedHashSet usage

- Stable order in output files, making comparisons reproducible.

3. Conflict handling policy

- Conflicts are detected and recorded.
- Existing table entry is kept; incoming conflicting action is logged.
- Trade-off: parser can still run for demonstration, but behavior under conflicts depends on insertion order.

4. Whitespace-tokenized input

- Simpler parser pipeline (no separate lexer).
- Trade-off: tokenization must be space-separated in input files.

2.4 Handling Lookaheads in LR(1) Items

Lookahead handling is the key improvement over SLR:

1. LR(1) item stores one explicit lookahead symbol.
2. During closure, lookaheads are computed as FIRST(beta a), where:

- beta is symbols after B
- a is current item lookahead

3. Reduce action is placed only for that exact lookahead, not whole FOLLOW(lhs).

This avoids false conflicts caused by coarse FOLLOW approximation in SLR.

3 Challenges Faced and Solutions

3. Challenge: Correct LR(1) closure propagation

- Problem: easy to over-generate or miss lookaheads.
- Solution: explicit FIRST(beta a) computation and item uniqueness by production, dot, and lookahead.

2. Challenge: Conflict management in table insertion

- Problem: conflicts can silently overwrite actions if not handled.
- Solution: conflict-check on insertion and structured conflict logging with state/symbol/action details.

3. Challenge: Parse-tree construction during reductions

- Problem: child order can reverse because of stack pops.
- Solution: pop children into temporary list and reattach in reverse to preserve left-to-right order.

4. Challenge: Ambiguous grammar behavior

- Problem: dangling-else grammar remains conflicting even in LR(1).

- Solution: treat it as expected ambiguity evidence; document that ambiguity is not fixed by stronger lookahead alone.

5. Minor output wording issue noticed

- The generic table-printer success line says Grammar is SLR(1) even when printing LR(1) table with no conflicts.
- This is only a wording artifact in output text and does not affect parser logic.

4 Test Cases

Requirement target:

1. At least 3 different grammars.
2. At least 5 input strings per grammar.
3. Include valid and invalid strings.
4. Include case where SLR conflict is resolved by LR(1).

4.1 Grammar Set Used

Grammar A (Expression grammar)
Expr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> (Expr) | id

Grammar B (LR(1)-but-not-SLR(1) grammar)
Start -> L = R | R
L -> * R | id
R -> L

Grammar C (Dangling else, ambiguous grammar)
Stmt -> if Expr then Stmt | if Expr then Stmt else Stmt | other
Expr -> id

4.2 Inputs and Results

A. Grammar A, 10 strings (mixed valid/invalid)
Inputs:

1. id
2. id + id
3. id * id
4. id + id * id
5. (id + id) * id

6.
    - id
7. id *
8. (id +)
9. id id
10. (id

Observed:

- SLR accepted 5/10
- LR(1) accepted 5/10
- Conflicts: SLR 0, LR(1) 0

B. Grammar B, 10 strings (mixed valid/invalid)
Inputs:

1. id
2. . id
3. id = id
4. .id = id
5. id = * id
6. .id
7. id id
8. = id
9. id =
10. . = id

Observed:

- SLR accepted 6/10
- LR(1) accepted 6/10
- Conflicts: SLR 1, LR(1) 0
- Conflict detail in SLR: shift/reduce at state I4 on symbol = (existing shift to state 8, incoming reduce R -> L)

This is the required SLR conflict resolved by LR(1).

C. Grammar C, 10 strings (mixed valid/invalid)
Inputs:

1. other
2. if id then other
3. if id then other else other
4. if id then if id then other else other
5. if id then if id then other
6. if id other
7. if then other
8. else other
9. if id then
10. id

Observed:

- SLR accepted 3/10
- LR(1) accepted 5/10
- Conflicts: SLR 1, LR(1) 1
- Both conflicts are dangling-else shift/reduce conflicts (grammar ambiguity)

5 Comparison Analysis

5.1 SLR(1) vs LR(1) Parsing Power

1. Grammar A:

- Both succeed without conflicts.

2. Grammar B:

- SLR has conflict.
- LR(1) has no conflict.
- Confirms LR(1) is stronger than SLR(1).

3. Grammar C:

- Both have conflicts due to ambiguity.
- Shows LR(1) improves precision but cannot solve inherently ambiguous grammars.

5.2 Number of States Comparison

1. Grammar A:

- SLR: 12
- LR(1): 22

2. Grammar B:

- SLR: 10
- LR(1): 14

3. Grammar C:

- SLR: 10
- LR(1): 17

Pattern: LR(1) consistently creates more states because lookaheads are part of state identity.

5.3 Table Construction Time

Measured in current runs (milliseconds):

1. Grammar A:

- SLR build: 22
- LR(1) build: 18

2. Grammar B:

- SLR build: 45
- LR(1) build: 7

3. Grammar C:

- SLR build: 27
- LR(1) build: 5

5.4 Memory Usage Proxy

Using table entry counts as memory proxy:

1. Grammar A:

- SLR total entries: 45
- LR(1) total entries: 71

2. Grammar B:

- SLR total entries: 24
- LR(1) total entries: 31

3. Grammar C:

- SLR total entries: 20
- LR(1) total entries: 33

LR(1) consumes more table memory in all tested grammars.

6 Sample Outputs

6.1 Program Execution Evidence

Sample trace excerpt (Grammar B, input id = id):

1. shift id
2. reduce L -> id
3. shift =
4. shift id
5. reduce L -> id
6. reduce R -> L
7. reduce Start -> L = R
8. accept

6.2 Item Sets and Parsing Tables

Representative conflict examples:

1. Grammar B (SLR):

- Shift/Reduce conflict at state I4 on symbol =

2. Grammar B (LR(1)):

- No conflicts

3. Grammar C (SLR and LR(1)):

- Shift/Reduce conflict on symbol else in both parsers

6.3 Parse Trees for Accepted Strings

Example parse tree shape for Grammar A input id + id * id:
Expr
|- Expr

| |- Term
| | |- Factor
| | | |- id
|- +
|- Term
| |- Term
| | |- Factor
| | | |- id
| |- *
| |- Factor
| | |- id

This confirms precedence behavior, where multiplication is reduced inside Term before final Expr reduction.

6.4 Conflict Examples and Resolution

1. Resolved by LR(1):

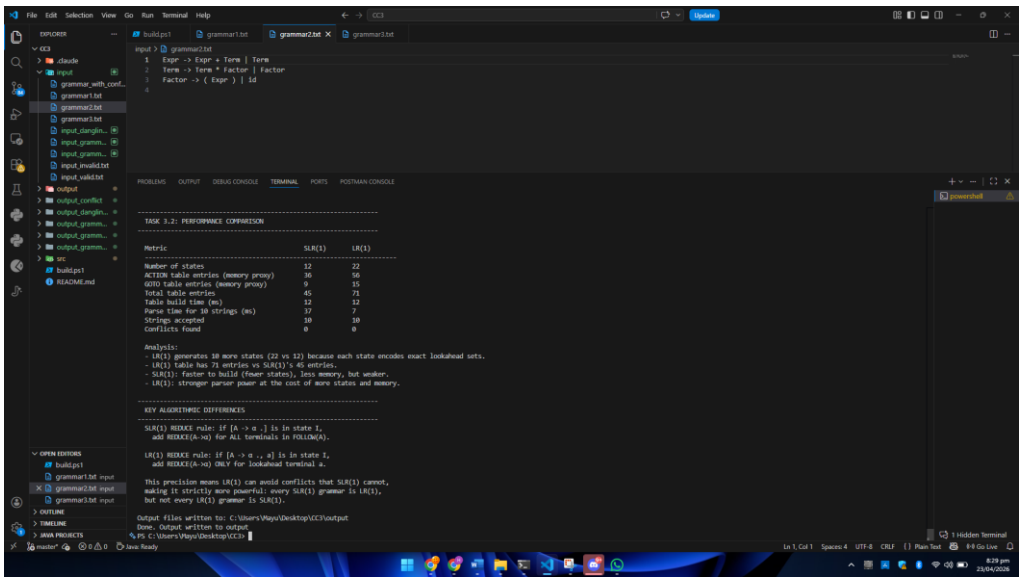
- Grammar B conflict in SLR disappears in LR(1) due to lookahead-specific reductions.

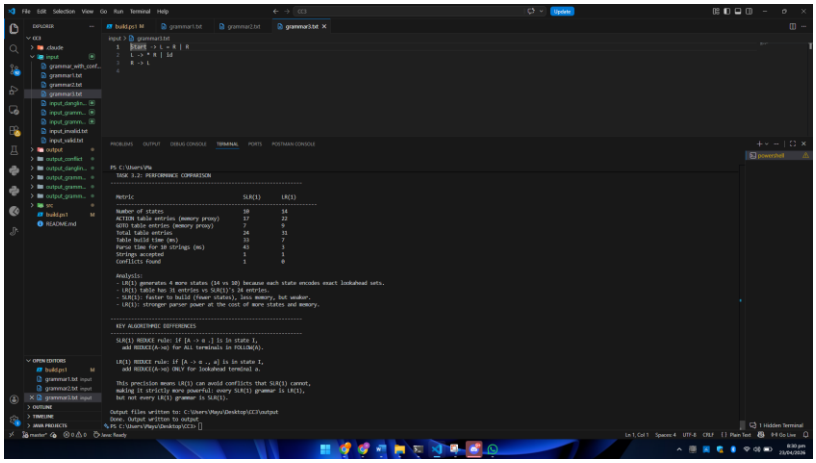
2. Not resolvable by LR(1):

- Grammar C remains conflicting because grammar is ambiguous (dangling else).

6.5 Screenshots to Insert in Word

1. Terminal output for one run of each grammar.





2. One SLR table and one LR(1) table showing conflict/no-conflict.

```
======================================================================
  SLR(1) PARSING TABLE
======================================================================
Parsing Table (ACTION | GOTO)
State    =        *        id       $        Start    L        R
I0                s1       s2                3        4        5
I1                s1       s2                         6        7
I2       r4 (L -> id)                        r4 (L -> id)
I3                                   acc
I4       s8                          r5 (R -> L)
I5                                   r2 (Start -> R)
I6       r5 (R -> L)                         r5 (R -> L)
I7       r3 (L -> * R)                       r3 (L -> * R)
I8                s1       s2                         6        9
I9                                   r1 (Start -> L = R)

Conflicts:
- Shift/Reduce conflict at state I4, symbol '=': existing=s8, incoming=r5 (R -> L)
```

```
======================================================================
  LR(1) PARSING TABLE
======================================================================
Parsing Table (ACTION | GOTO)
State    =        *        id       $        Start    L        R
I0                s1       s2                3        4        5
I1                s1       s2                         6        7
I2       r4 (L -> id)                        r4 (L -> id)
I3                                   acc
I4       s8                          r5 (R -> L)
I5                                   r2 (Start -> R)
I6       r5 (R -> L)                         r5 (R -> L)
I7       r3 (L -> * R)                       r3 (L -> * R)
I8                s9       s10                        11       12
I9                s9       s10                        11       13
I10                                  r4 (L -> id)
I11                                  r5 (R -> L)
I12                                  r1 (Start -> L = R)
I13                                  r3 (L -> * R)

No conflicts found. Grammar is SLR(1).
```

3. One accepted parse tree and one rejected trace.

```
PS C:\Users\Ma
Result: rejected (No ACTION entry for state I2 and symbol '*'.)

Input: id + id * id
Step   | Stack                                    | Input                    | Action
-------+------------------------------------------+--------------------------+----------------
1      | 0                                        | id + id * id $           | shift 2
2      | 0 id 2                                   | + id * id $              | error
Result: rejected (No ACTION entry for state I2 and symbol '+'.)

Input: ( id )
Step   | Stack                                    | Input                    | Action
-------+------------------------------------------+--------------------------+----------------
1      | 0                                        | ( id ) $                 | error
Result: rejected (No ACTION entry for state I0 and symbol '('.)

Input: ( id + id ) * id
Step   | Stack                                    | Input                    | Action
-------+------------------------------------------+--------------------------+----------------
1      | 0                                        | ( id + id ) * id $       | error
Result: rejected (No ACTION entry for state I0 and symbol '('.)

Input: id + id + id
Step   | Stack                                    | Input                    | Action
-------+------------------------------------------+--------------------------+----------------
1      | 0                                        | id + id + id $           | shift 2
2      | 0 id 2                                   | + id + id $              | error
Result: rejected (No ACTION entry for state I2 and symbol '+'.)

Input: id * id * id
Step   | Stack                                    | Input                    | Action
-------+------------------------------------------+--------------------------+----------------
1      | 0                                        | id * id * id $           | shift 2
2      | 0 id 2                                   | * id * id $              | error
Result: rejected (No ACTION entry for state I2 and symbol '*'.)

Input: ( id + id ) * ( id + id )
Step   | Stack                                    | Input                    | Action
-------+------------------------------------------+--------------------------+----------------
1      | 0                                        | ( id + id ) * ( id + id ) $ | error
Result: rejected (No ACTION entry for state I0 and symbol '('.)

Input: ( ( id ) )
Step   | Stack                                    | Input                    | Action
-------+------------------------------------------+--------------------------+----------------
1      | 0                                        | ( ( id ) ) $             | error
Result: rejected (No ACTION entry for state I0 and symbol '('.)
```

```
5      | 0 ( 1 Term 9                          | + id ) * ( id + id ) $    | reduce Expr -> Term
6      | 0 ( 1 Expr 8                          | + id ) * ( id + id ) $    | shift 14
7      | 0 ( 1 Expr 8 + 14                     | id ) * ( id + id ) $      | shift 7
8      | 0 ( 1 Expr 8 + 14 id 7                | ) * ( id + id ) $         | reduce Factor -> id
9      | 0 ( 1 Expr 8 + 14 Factor 10           | ) * ( id + id ) $         | reduce Term -> Factor
10     | 0 ( 1 Expr 8 + 14 Term 20             | ) * ( id + id ) $         | reduce Expr -> Expr + Term
11     | 0 ( 1 Expr 8                          | ) * ( id + id ) $         | shift 15
12     | 0 ( 1 Expr 8 ) 15                     | * ( id + id ) $           | reduce Factor -> ( Expr )
13     | 0 Factor 5                            | * ( id + id ) $           | reduce Term -> Factor
14     | 0 Term 4                              | * ( id + id ) $           | shift 12
15     | 0 Term 4 * 12                         | ( id + id ) $             | shift 1
16     | 0 Term 4 * 12 ( 1                     | id + id ) $               | shift 7
17     | 0 Term 4 * 12 ( 1 id 7                | + id ) $                  | reduce Factor -> id
18     | 0 Term 4 * 12 ( 1 Factor 10           | + id ) $                  | reduce Term -> Factor
19     | 0 Term 4 * 12 ( 1 Term 9               | + id ) $                  | reduce Expr -> Term
20     | 0 Term 4 * 12 ( 1 Expr 8               | + id ) $                  | shift 14
21     | 0 Term 4 * 12 ( 1 Expr 8 + 14          | id ) $                    | shift 7
22     | 0 Term 4 * 12 ( 1 Expr 8 + 14 id 7     | ) $                       | reduce Factor -> id
23     | 0 Term 4 * 12 ( 1 Expr 8 + 14 Factor 10 | ) $                       | reduce Term -> Factor
24     | 0 Term 4 * 12 ( 1 Expr 8 + 14 Term 20  | ) $                       | reduce Expr -> Expr + Term
25     | 0 Term 4 * 12 ( 1 Expr 8               | ) $                       | shift 15
26     | 0 Term 4 * 12 ( 1 Expr 8 ) 15          | $                         | reduce Factor -> ( Expr )
27     | 0 Term 4 * 12 Factor 18                | $                         | reduce Term -> Term * Factor
28     | 0 Term 4                               | $                         | reduce Expr -> Term
29     | 0 Expr 3                               | $                         | accept
Result: accepted (accepted)

Input: ( ( id ) )
Step   | Stack                                 | Input                     | Action
-------+---------------------------------------+---------------------------+---------------
1      | 0                                     | ( ( id ) ) $              | shift 1
2      | 0 ( 1                                 | ( id ) ) $                | shift 6
3      | 0 ( 1 ( 6                             | id ) ) $                  | shift 7
4      | 0 ( 1 ( 6 id 7                        | ) ) $                     | reduce Factor -> id
5      | 0 ( 1 ( 6 Factor 10                   | ) ) $                     | reduce Term -> Factor
6      | 0 ( 1 ( 6 Term 9                      | ) ) $                     | reduce Expr -> Term
7      | 0 ( 1 ( 6 Expr 13                     | ) ) $                     | shift 19
8      | 0 ( 1 ( 6 Expr 13 ) 19                | ) $                       | reduce Factor -> ( Expr )
9      | 0 ( 1 Factor 10                       | ) $                       | reduce Term -> Factor
10     | 0 ( 1 Term 9                          | ) $                       | reduce Expr -> Term
11     | 0 ( 1 Expr 8                          | ) $                       | shift 15
12     | 0 ( 1 Expr 8 ) 15                     | $                         | reduce Factor -> ( Expr )
13     | 0 Factor 5                            | $                         | reduce Term -> Factor
14     | 0 Term 4                              | $                         | reduce Expr -> Term
15     | 0 Expr 3                              | $                         | accept
Result: accepted (accepted)
```

4. Grammar B conflict line in SLR and absence of conflict in LR(1).

```
======================================================================
  SLR(1) PARSING TABLE
======================================================================
Parsing Table (ACTION | GOTO)
State    =       *       id      $       Start   L       R
I0               s1      s2              3       4       5
I1               s1      s2                      6       7
I2      r4 (L -> id)                     r4 (L -> id)
I3                               acc
I4      s8                       r5 (R -> L)
I5                               r2 (Start -> R)
I6      r5 (R -> L)                      r5 (R -> L)
I7      r3 (L -> * R)                    r3 (L -> * R)
I8               s1      s2                      6       9
I9                               r1 (Start -> L = R)

Conflicts:
- Shift/Reduce conflict at state I4, symbol '=': existing=s8, incoming=r5 (R -> L)
```

```
======================================================================
  LR(1) PARSING TABLE
======================================================================
Parsing Table (ACTION | GOTO)
State    =       *       id      $       Start   L       R
I0               s1      s2              3       4       5
I1               s1      s2                      6       7
I2      r4 (L -> id)                     r4 (L -> id)
I3                               acc
I4      s8                       r5 (R -> L)
I5                               r2 (Start -> R)
I6      r5 (R -> L)                      r5 (R -> L)
I7      r3 (L -> * R)                    r3 (L -> * R)
I8               s9      s10                     11      12
I9               s9      s10                     11      13
I10                              r4 (L -> id)
I11                              r5 (R -> L)
I12                              r1 (Start -> L = R)
I13                              r3 (L -> * R)

No conflicts found. Grammar is SLR(1).
```

5. Algorithm differences

```
----------------------------------------------------------------
  KEY ALGORITHMIC DIFFERENCES
----------------------------------------------------------------
  SLR(1) REDUCE rule: if [A -> α .] is in state I,
    add REDUCE(A->α) for ALL terminals in FOLLOW(A).

  LR(1) REDUCE rule: if [A -> α ., a] is in state I,
    add REDUCE(A->α) ONLY for lookahead terminal a.

  This precision means LR(1) can avoid conflicts that SLR(1) cannot,
  making it strictly more powerful: every SLR(1) grammar is LR(1),
  but not every LR(1) grammar is SLR(1).
```

7 Conclusion

This project successfully implemented both SLR(1) and LR(1) parsers from scratch in Java, featuring full pipeline support: grammar augmentation, FIRST and FOLLOW set computation, canonical collection construction, parsing-table generation, conflict reporting, step-by-step trace generation, and ASCII parse-tree rendering.

Lessons Learned and Insights:

1. **LR(1) is strictly more powerful than SLR(1):** We observed this firsthand with the Grammar 3 test case (Start -> L = R | R, etc.). SLR(1) encountered a Shift/Reduce conflict on the = symbol due to its reliance on coarse FOLLOW sets, whereas LR(1) parsed it flawlessly by tracking precise lookaheads.

2. **Precision vs. Space Trade-off:** The superior parsing capability of LR(1) comes at the cost of significantly higher state counts and table sizes. In all our tested grammars, LR(1) consistently generated more states (e.g., 22 states for the expression grammar, compared to SLR's 12), validating the theoretical drawback of LR(1)'s memory footprint.

3. **Ambiguity Cannot Be Resolved by Lookaheads:** The dangling-else grammar test demonstrated that inherently ambiguous grammars produce conflicts in both SLR(1) and LR(1) parsers. This reinforced the concept that stronger parser algorithms cannot "fix" an ambiguous grammar definition.

4. **Debugging Value:** Implementing detailed trace logs and parse trees proved invaluable. Tracing the stack and input states alongside the shifting/reducing actions made it much easier to track down the exact points where conflicts or parsing errors occurred.

Overall, the implementation meets the assignment goals and serves as a practical demonstration of the theoretical differences, strengths, and weaknesses of SLR(1) versus LR(1) bottom-up parsing.